

用第一性原理拆解 Agentic Coding：从理论到实操（上）

本文作者：小夏，TRAE 技术专家

📌 当我们使用 AI 编程助手时，很容易把它当作一个「黑盒」：输入我们的需求，就可以获得想要的代码。但如果想真正高效地利用这些工具，我们需要回到第一性原理：理解 LLM 究竟是如何工作的，它的能力边界在哪里，以及 Agent 架构是如何在这些约束下被设计出来的。在本文上篇，我们将从大模型的工作原理和局限性出发，介绍由此引发的 Coding Agent 中的效果问题，进而支撑我们理解各种最佳实践背后的原因，并且你会发现，很多 TRAE 中的 Agent 优化和功能特性本质上都是在解决这些大模型固有的局限性。

第一性原理 | 理解 LLM 的本质

LLM 是如何「思考」的

大语言模型的工作方式可以用一句话概括：**预测下一个 token。**

当你向 LLM 提问时，它并不是先在「脑中」构思好完整答案再输出。相反，它每次只做一件事：基于当前看到的所有文本（你的输入 + 它已经生成的内容），预测最可能的下一个词是什么。然后把这个词加入序列，再预测下一个，如此循环直到生成结束，这种方式被称为 **自回归生成（Autoregressive Generation）**。

理解这一点至关重要，因为它揭示了 LLM 的几个本质特征：

- 没有独立于输出的「思考」过程：**LLM 的推理就是生成本身。它不能先想好再说，只能边说边想。这就是为什么让模型「一步步思考」（Chain-of-Thought）会提升效果 — 你实际上是在给它更多的「思考空间」，让它通过生成中间步骤来辅助推理。尽管新一代模型很多都带有显式的 reasoning 能力，依然脱离不了这个范式。
- 上下文就是全部记忆：**LLM 没有独立的记忆存储。它对当前对话的所有「理解」，完全来自于输入给它的上下文窗口。窗口之外的内容，对模型来说就是不存在的。
- 生成具有概率性：**每一步预测都是基于概率分布采样，这意味着同样的输入可能产生不同的输出。

自回归（autoregressive）LLM 的核心工作方式是：**一次只预测下一个 token**，然后把自己刚生成的内容再喂回去继续生成。这个机制很强（通用、可扩展、能「写出」复杂结构），但也天然带来一些

局限，在 Coding Agent 里这些局限会被放大，因为软件开发是「长链路、强约束、强验证」的任务。

当前 Coding Agent 普遍存在的问题：

1. 局部最优：一步一步「续写」，不等于全局规划

- 容易先写得很像、很顺，然后在后面发现不对再补丁式修修补补（历史债务快速累积）。
- 多文件重构/架构调整这种「先规划后落地」的任务，会变成边写边想，导致接口不一致、重复实现、遗漏迁移步骤。
- 对「最终能不能通过测试/CI」缺乏感知，除非你把测试结果显式喂回 agent 循环。

2. 一旦走偏，会沿着偏差继续滚雪球

- 早期误读需求/误判代码库的约束，导致后面每一步都在错误世界观里自治。
- 生成了一个不存在的函数名/类型后，后续会越来越「相信它存在」，直到编译/运行时打脸。
- 在长对话、长任务里尤其明显：越写越像「另一个项目」。

3. 输出是流式的，无法回头修改之前输出的内容

- 擅长「写新文件/新段落」，相对不擅长在复杂代码中做精确、最小化、局部一致的编辑。
- 容易倾向于「重写一遍」而不是「进行修改」，导致 diff 过大、review 成本高。
- 在需要保持大量不变量（API、格式、风格、兼容性）的仓库里尤其痛苦。

4. 满足约束的能力有限，而代码是强语法/强语义约束的

- 「看起来合理」的实现，经常在边界条件、并发、错误处理、资源释放上出错。
- 跨文件的符号解析、泛型/模板推导、宏/代码生成等，靠纯文本推断很容易漏。
- 复杂的重构（rename、move、split）没有 AST/语义工具辅助时，错一个引用就全盘崩。

5. 天然单线程思考，难并行探索

- 在方案选择（架构、依赖、实现路径）上容易早早押注一个方向，缺少系统性对比。
- 排查 bug 时更像「单线推理」，不如人类/工具那样并行假设、并行验证

Attention：LLM 如何「阅读」上下文

在预测下一个 token 时，模型需要从上下文中提取相关信息，这个过程由 **Attention（注意力）机制** 完成。

你可以把 Attention 想象成一个动态的「聚光灯」：当模型生成每个新 token 时，它会扫视整个上下文，对不同位置的内容分配不同的「注意力权重」。权重高的部分会对当前生成产生更大影响，权重低的部分则几乎被忽略。

具体来说，对于当前要生成的 token，模型会：

- **计算相关性**：当前位置与上下文中每个位置的相关程度
- **分配权重**：相关性高的位置获得更高权重，形成一个概率分布
- **加权聚合**：根据权重汇总上下文信息，用于预测下一个 token

这个机制有几个重要特性：

- **注意力是稀疏的**：模型不会均匀地关注所有内容。在实践中，attention 权重往往集中在少数关键位置，大部分位置的权重接近于零。
- **注意力需要「学习」的**：模型通过训练学习在什么情况下关注什么内容。这意味着它可能会形成某些偏好模式，比如更关注开头和结尾。
- **计算复杂度是平方级的**：标准 Attention 需要计算每个位置与所有其他位置的关系，计算量随上下文长度的平方增长。这是上下文长度存在物理上限的根本原因之一。

理解了 Attention 机制，我们就能更好地理解上下文长度为什么会带来多种限制。

上下文长度：最关键的约束

上下文窗口的限制是影响 Coding Agent 设计最核心的约束条件。结合对 Attention 机制的理解，我们可以更清楚地看到这个限制的多个层次。

物理上限

每个模型都有最大 token 限制，目前主流模型通常在 128K 到 200K 之间。这个硬性边界的存在有两个根本原因：

1. Attention 的计算复杂度通常是 $O(n^2)$ ，上下文长度翻倍，计算量变成四倍
2. 需要存储完整的注意力矩阵，内存消耗同样是平方级增长

超过这个长度的内容根本无法被处理。

有效上下文远小于标称上下文

虽然模型宣称支持 200K tokens，但这并不意味着在 200K 长度下都能保持良好表现。

问题出在 Attention 的「稀疏性」上：当上下文很长时，注意力权重需要分散到更多位置。如果关键信息只是海量内容中的一小部分，它获得的注意力权重可能会被稀释到难以有效利用的程度。实际上，一个支持 200K 的模型，可能在超过 80K 或 100K 之后，就开始出现明显的性能退化。

性能退化曲线

随着上下文变长，不仅是「记忆」变差，模型的整体能力，包括推理准确性、指令遵循能力、代码生成质量都会下降。

这是自回归 + Attention 组合带来的累积效应：

- 每一步生成都依赖于对上下文的正确理解
- 更长的上下文意味着更稀疏的注意力分布

- 错误理解会传递到后续生成，形成累积误差

理解这些限制，你就能明白为什么 Coding Agent 需要精心设计上下文管理策略，而不是简单地把所有信息塞进窗口。

强化学习：让模型学会「做事」

前面我们讨论的是 LLM 的基础能力，预测下一个 token。但要让模型从「能说会道」进化到「能做事情」，还需要另一个关键技术：**强化学习（Reinforcement Learning，RL）**。

• 预训练的局限

LLM 的预训练本质上是「模仿」：通过阅读海量文本，学会预测「人类会怎么写下一个词」。这让模型获得了广泛的知识和流畅的表达能力，但也有明显的局限：

- 模型学会的是「文本看起来应该是什么样」，而不是「什么行动能解决问题」
- 预训练数据中很少有「调用工具 → 观察结果 → 调整策略」这样的交互序列
- 模型可能会生成看起来合理但实际无效的操作步骤

简单来说，预训练教会了模型「说」，但没有教会它「做」。

• 强化学习的核心思想

强化学习用一个简单的循环来解决这个问题：**尝试 → 反馈 → 调整**。

想象教一个小朋友下棋：你不会给他看一百万盘棋谱让他背（这是预训练的方式），而是让他实际下棋，赢了就表扬，输了就复盘。通过无数次的尝试和反馈，他逐渐学会什么是好棋、什么是坏棋。

对 LLM 来说，强化学习的过程类似：

- **尝试**：让模型在真实或模拟的环境中执行任务（比如调用工具、编辑文件）
- **反馈**：根据结果给出奖励信号（任务完成了？代码能运行？测试通过了？）
- **调整**：更新模型参数，让它更倾向于选择能获得高奖励的行动

这个过程会重复成千上万次，模型逐渐学会：在什么情况下应该调用什么工具、如何解读工具返回的结果、什么时候应该继续尝试、什么时候应该换个方向。

为什么 RL 对 Agentic 能力至关重要

传统的 LLM 微调（比如监督学习）需要人类标注「正确答案」。但 Agent 任务的特点是：

- **路径多样**：完成同一个编程任务可能有无数种合理的步骤组合
- **结果导向**：我们真正关心的是最终结果，而不是中间每一步是否「标准」
- **需要探索**：有时候必须尝试几种方法才能找到可行的路径

RL 天然适合这种场景。它不要求你定义「正确的步骤序列」，只需要定义「什么算成功」。模型通过自己的探索，学会找到通往成功的路径。

实际训练中的应用

现代 Coding Agent 背后的模型通常会经历这样的 RL 训练：

- 在模拟的编程环境中执行任务（创建文件、运行代码、修复 bug）
- 用测试通过率、代码质量评分等作为奖励信号
- 学习何时应该读取更多文件来获取上下文，何时应该直接动手修改
- 学习如何从错误信息中提取有用线索，调整下一步策略

值得注意的是，RL 训练的效果高度依赖奖励信号的设计。如果只奖励「任务完成」，模型可能学会走捷径（Reward Hacking）；如果奖励信号太复杂，训练可能不稳定。**这也是为什么不同模型在 Agent 任务上的表现差异很大，背后的 RL 训练策略可能完全不同。**

对使用者的启示

理解了 RL 的作用，你会更清楚为什么某些使用方式更有效：

- **提供清晰的成功标准：**当你告诉 Agent 「修好这个 bug，运行 npm test 应该全部通过」，你实际上是在给它一个明确的「奖励信号」，这与它训练时的模式一致
- **允许试错：**Agent 在训练中学会了通过尝试来解决问题，给它多次尝试的机会往往比期待一次成功更实际
- **观察它的决策模式：**当 Agent 做出某个决定（比如先读文件而不是直接修改），这往往反映了它在 RL 训练中学到的策略

Coding Agent 的实现原理

理解了 LLM 的本质和限制后，我们来看 Coding Agent 是如何在这些约束下被设计出来的。

LLM API 的核心结构

现代 LLM API 采用基于消息的对话结构。理解这个结构是构建 Agent 的基础。

Messages 数组

API 的核心是一个消息数组，每条消息包含角色（role）和内容（content）：

代码块

```
1  {
2    "messages": [
3      {
4        "role": "system",
5        "content": "你是一个专业的编程助手..."
6      },
7      {
8        "role": "user",
9        "content": "帮我写一个快速排序函数"
10     },

```

```

11     {
12         "role": "assistant",
13         "content": "好的，这是一个 Python 实现的快速排序..."
14     }
15 ]
16 }

```

三种角色各有用途：

- **system**：设定 Agent 的行为准则、能力范围和工具定义
- **user**：用户的输入
- **assistant**：模型的回复

Tool Calling 机制

Coding Agent 的核心能力来自工具调用。API 允许你定义工具的 schema，模型会在需要时生成结构化的工具调用请求：

代码块

```

1  {
2      "tools": [
3          {
4              "name": "read_file",
5              "description": "读取指定路径的文件内容",
6              "parameters": {
7                  "type": "object",
8                  "properties": {
9                      "path": {
10                         "type": "string",
11                         "description": "文件路径"
12                     }
13                 },
14                 "required": ["path"]
15             }
16         }
17     ]
18 }

```

当模型决定调用工具时，它的响应会包含工具调用信息：

代码块

```

1  {
2      "role": "assistant",
3      "content": null,

```

```
4     "tool_calls": [  
5         {  
6             "id": "call_abc123",  
7             "name": "read_file",  
8             "arguments": "{\"path\": \"src/main.py\"}"  
9         }  
10    ]  
11 }
```

工具执行结果作为新消息返回给模型：

代码块

```
1  {  
2      "role": "tool",  
3      "tool_call_id": "call_abc123",  
4      "content": "def main():\n    print('Hello, World!')\n..."  
5  }
```

Reasoning Content 的保留

对于具有显式推理能力的模型，API 响应中可能包含 reasoning 或 thinking 字段，记录模型的思考过程：

代码块

```
1  {  
2      "role": "assistant",  
3      "reasoning_content": "用户需要读取文件来理解项目结构。我应该先查看 src 目录下的主要文件...",  
4      "content": "让我先看一下项目的主文件。",  
5      "tool_calls": [...]  
6  }
```

在多轮对话中保留这些推理内容，可以帮助模型维持思维的连贯性。模型可以「回顾」自己之前的思考过程，从而做出更一致的决策。

Prompt Caching：工程实践的关键

由于 LLM 的自回归特性，每次请求都需要重新处理整个上下文。对于动辄几万 token 的 Coding Agent 对话来说，这意味着大量的重复计算和延迟。

Prompt Caching 通过缓存上下文前缀的计算结果来解决这个问题。关键在于：**缓存基于前缀匹配**。只有当新请求的开头部分与之前的请求完全一致时，才能命中缓存。

这直接影响了 prompt 的组织方式：



最佳实践参考：

- 1. 把稳定内容放在前面：system prompt 和工具定义应该保持稳定，避免频繁修改
- 2. 动态内容放在后面：对话历史和当前输入放在最后
- 3. 避免在稳定前缀中插入可变内容：比如不要在 system prompt 中插入当前时间戳

Agent Loop：核心循环

语言模型可以回答问题，而 Agent 可以**做事情**。Agent Loop 正是实现这一差异的关键。

当模型收到一个无法仅凭训练知识完成的请求时，它需要与外部世界交互：读取文件、查询数据库、执行代码。Agent Loop 就是管理这个「推理-行动」循环的编排层，使模型能够处理需要多个步骤、外部信息或产生实际影响的任务。

循环的基本原理

Agent Loop 的运作遵循一个简单的原则：调用模型 → 检查是否需要使用工具 → 如果需要则执行工具 → 将结果返回给模型再次调用 → 重复直到模型产生最终响应。

这个循环的关键在于**上下文的累积**。每次迭代都会向对话历史中添加新内容。模型不仅能看到原始请求，还能看到它调用过的每个工具以及收到的每个结果。这种累积的上下文使得复杂的多步骤推理成为可能。

一个具体的例子

假设用户请求：「帮我修复项目中的这个 bug：用户登录后会话没有正确保存」。

这不是模型仅凭知识就能完成的任务，它需要通过 Agent Loop 逐步探索：

1. **第一轮**：模型收到请求，首先需要了解项目结构。它调用 `list_directory` 工具查看项目根目录。
2. **第二轮**：模型看到了目录结构，识别出 `src/auth/` 目录可能与登录相关。它调用 `read_file` 查看 `src/auth/login.js`。
3. **第三轮**：模型看到了登录代码，发现它调用了 `sessionManager.save()`。为了追踪问题，它调用 `read_file` 查看 `src/session/manager.js`。
4. **第四轮**：模型发现了问题，`save()` 方法中有一个异步操作没有被正确 `await`。它调用 `edit_file` 工具修复这个 bug。
5. **第五轮**：修复完成，模型调用 `shell` 工具运行测试来验证修复是否有效。
6. **第六轮**：测试通过。模型不再请求工具，而是生成最终响应：总结问题原因、修复内容和验证结果。

每一轮都遵循相同的模式：模型接收上下文，决定是继续行动还是给出响应，要么继续循环，要么退出。关键在于，模型基于对任务不断演进的理解**自主做出这些决策**。

循环的终止条件

每次模型调用都会返回一个停止原因（stop reason），决定接下来发生什么：

- **end_turn**：模型完成了响应，没有进一步的行动需要执行。这是正常的成功终止，循环退出并返回最终消息。
- **tool_use**：模型想要执行一个或多个工具后再继续。循环执行请求的工具，将结果追加到对话历史，然后再次调用模型。
- **max_tokens**：模型的响应因达到 token 限制而被截断。这在当前循环中无法恢复，循环以错误终止。

理解这些终止条件有助于预测 Agent 的行为并处理边界情况。

Coding Agent 的典型工具

一个功能完整的 Coding Agent 通常需要以下几类工具：

文件操作

- `read_file`：读取文件内容
- `write_file`：创建或覆盖文件
- `edit_file`：对文件进行局部编辑（而非完全重写）

代码执行

- `shell` / `terminal`：执行命令行命令，用于运行代码、安装依赖、执行测试等

代码搜索

- `grep` / `search`：在代码库中搜索文本或模式
- `semantic_search`：基于语义的代码搜索

项目导航

- `list_directory`：列出目录内容
- `find_files`：根据模式查找文件

这些工具的设计直接影响 Agent 的能力边界，工具的粒度、参数设计、返回格式都需要仔细考量。

常见问题与解决方案

在了解了 LLM 和 Coding Agent 的基本原理后，我们再来看看一些常见的问题就能更好地理解。

会话间失忆

Coding Agent 面临的一个根本性问题是：**它们在会话之间没有持久记忆。**

每次启动新会话时，Agent 只知道它能在磁盘上找到的内容。就像电影《记忆碎片》或《初恋 50 次》中的主角一样，Agent 每天醒来都不记得昨天发生了什么。而典型的工程 workflow 往往需要跨越多个会话才能完成一个功能——因为需要测试、代码审查和后续清理。

这导致了一个荒谬的局面：你需要在每个新会话开始时，重新向 Agent 解释项目背景、当前进度和接下来的计划。

解决方案：

- 使用结构化的任务追踪系统（如 issue tracker）
- 在每个会话结束时，让 Agent 生成一个状态摘要，供下次会话使用
- 将重要决策和发现记录在固定位置，而不是对话历史中

上下文窗口耗尽

每轮循环都会向对话历史添加消息。对于需要多次工具调用的复杂任务，历史记录可能超出模型的上下文窗口。当这种情况发生时，Agent 无法继续。

具体表现包括：模型提供商返回输入长度错误，或随着上下文填满不太相关的早期消息，模型性能明显下降。

目前有两种主流的上下文管理策略：

Observation Masking（观察遮蔽）

这种方法只针对工具返回的观察结果进行处理，而完整保留 Agent 的推理和行动历史。具体做法是：用占位符替换较早轮次的观察内容（比如「内容已省略」），只保留最近 N 轮的完整输出。

这种方法简单高效，因为典型 Coding Agent 的每轮交互中，工具输出（如文件内容、命令执行结果）往往占据了绝大部分 token。通过遮蔽旧的观察内容，Agent 仍然可以访问自己过去的推理和决策，但不再重复处理早期轮次的冗长文本。

💡 LLM Summarization (LLM 摘要)

这种方法使用另一个 LLM 将较早的交互历史（包括观察、行动和推理）压缩成简短的摘要。摘要会替代原始的详细历史，而最近的几轮对话保持完整。

这种方法理论上可以支持无限长的对话，因为历史会被反复压缩。但它也有代价：每次生成摘要都需要额外的 API 调用，而且摘要可能会丢失某些细节信息。研究发现，摘要有时会掩盖 Agent 应该停止尝试的信号，导致 Agent 运行更多轮次，反而增加了成本。

两种方法各有优劣：Observation Masking 实现简单、成本低，但上下文仍会缓慢增长；LLM Summarization 可以更彻底地压缩历史，但引入了额外开销和信息损失风险。实践中，可以考虑混合使用，以 Observation Masking 作为主要策略，仅在上下文确实过长时触发 LLM Summarization。

其他解决方案：

- 减少工具输出的冗长程度，返回摘要或相关片段，而非完整数据
- 简化工具 schema，深度嵌套的结构会在工具配置和模型推理中消耗大量 token
- 将大型任务分解为子任务，每个子任务使用新的上下文

有效上下文远小于标称值

即使拥有 1M token 的上下文窗口，Coding Agent 实际上只能有效利用其中的 10-15%。超过 20% 后，成本和性能都会急剧恶化。大多数 Agent 会在达到约 20% 时强制中断，而最佳实践是在 15% 之前就重启会话。

这意味着，以全速工作时，你大约只有 5-10 分钟的有效工作时间，然后 Agent 就会「耗尽上下文」，需要重启（相当于死亡）或进行 compaction（相当于记忆清除）。

有一个生动的比喻：上下文窗口就像潜水员的氧气罐。所有人都说「我们给他一个更大的氧气罐：100 万 token！」但他最终还是会耗尽氧气，更大的窗口并不能解决根本问题。

「Dumb Zone」：中间区域的性能退化

研究发现，上下文窗口的中间 40-60% 区域存在一个「Dumb Zone」，在这个区域，模型的召回率下降，推理能力变差。这与前面提到的「Lost in the Middle」现象相呼应。

当 Agent 深入工作时，它会逐渐表现出类似「痴呆」的症状：迷失方向、产生幻觉接口、忘记原始目标。这不是因为模型变笨了，而是因为关键信息被淹没在大量的上下文中，无法被有效利用。

新发现的任务被丢弃

这是 Agentic Coding 中一个容易被忽视但影响巨大的问题：**LLM 在工作过程中会注意到各种问题，但在上下文空间紧张时，会选择忽略这些发现，不采取任何行动。**

例如，Agent 在修复一个 bug 时，可能会注意到代码中的另一个潜在问题、一个需要重构的地方、或一个缺失的测试用例。但如果当前上下文已经很满，Agent 可能会「假装没看到」，继续专注于手头的任务。这些被发现但被丢弃的工作，就这样悄悄消失了。

💡 解决方案：

- 使用外部工具让 Agent 随时记录发现的问题
- 在任务结束时，要求 Agent 列出所有观察到但未处理的问题
- 建立「发现即记录」的工作流程

过早宣告完成

当 Agent 经历多次 compaction 后，可能会出现一个荒谬的情况：它会自信地宣布「恭喜，任务完成了！让我们开始手动测试吧！」而实际上还有大量的阶段没有完成。

解决方案：

- 将整体计划保存在上下文之外的固定位置
- 定期让 Agent 对照原始计划检查进度
- 使用结构化的任务追踪，而不是依赖 Agent 的记忆

工具选择不当

当模型持续选择错误的工具时，问题通常出在工具描述的模糊性上。从模型的角度审视描述：如果两个工具的描述有重叠，模型就没有选择的依据，我们应该确保每个工具的用途清晰且互不重叠。

结语：从理解到行动的桥梁

通过上述的探讨，我们已经完成了从 LLM 本质到 Agentic Coding 原理的理论铺垫：从逐个预测 token 的底层逻辑，到注意力机制的上下文约束，再到强化学习如何让模型学会「做事」；从 API 结构的基础框架，到 Agent Loop 的核心循环，最后到解决上下文耗尽、会话失忆等工程问题的实践策略。这些内容共同构成了 Agentic Coding 的第一性原理基石——理解这些，你就能跳出“黑盒工具”的使用层面，看到各种最佳实践背后的深层逻辑。

在下篇中，我们将从理论走向实操：结合具体案例拆解如何设计高效的 Agent 对话流程、如何选择合适的工具集、如何优化上下文管理策略，以及如何将这些原理应用到实际的编码任务中（如 bug 修复、功能开发、代码重构）。你将看到，当我们用第一性原理的视角重新审视 Agentic Coding 时，那些看似零散的技巧会形成一套系统的方法论，帮助你真正成为 AI 编程助手的「驾驭者」而非「使用者」。

期待在下篇与你继续深入，一起把理论转化为可落地的高效实践。

更多信息请关注TRAEE企业版服务号 



微信搜一搜



TRAE企业版